

RESEARCH

Open Access



Improved polytopic differential neural distinguishers for SIMON, SIMECK, and SPECK block ciphers

Iman Mirzaali¹, Sadegh Sadeghi² and Nasour Bagheri^{1,3*} 

Abstract

In recent years, the application of deep learning in cryptanalysis has gained significant attention, particularly with the emergence of neural network-based distinguishers. At CRYPTO'19, Gohr demonstrated that neural networks could develop differential distinguishers capable of producing highly competitive attacks against existing methods. Building on this foundation, we propose multiple input polytopic differential neural distinguishers (*PDNDs*) for the lightweight block ciphers SIMON, SIMECK, and SPECK. Our approach incorporates a novel data generation method that utilizes two polytope differences, resulting in more precise training data and enhanced model accuracy. Through extensive experiments in single-key and related-key scenarios, we evaluate and validate the intrinsic performance of our neural distinguishers. Our results show that *PDNDs* significantly outperform the baseline, polytopic, multiple, and mixture differential neural distinguishers, utilizing a single input difference, in accuracy across various cipher rounds. Notably, our *PDNDs* achieved 100% accuracy for up to 7 rounds of SIMON32 and SIMECK32, and 99.39% accuracy for 5 rounds of SPECK32 in the single-key scenario. Additionally, for extended rounds, we achieved accuracy levels of up to 12 rounds for SIMON32, 13 for SIMECK32, and 8 for SPECK32 without requiring staged training. In the related-key scenario, our method further improved performance, introducing 13-round and 15-round *RK-PDNDs* for SIMON32 and SIMECK32, respectively, underscoring the enhanced capabilities of our approach. Furthermore, we demonstrate the effectiveness of our neural distinguishers through a key recovery test, where they successfully distinguish between correct and incorrect keys, confirming the practical applicability of our approach in cryptanalysis.

Keywords Block cipher, Polytopic differential cryptanalysis, Deep learning, Neural distinguisher

Introduction

The advancement of machine learning has profoundly impacted various aspects of people's lives. Among the various branches of machine learning, deep learning

has excelled particularly in a wide range of tasks across numerous domains. Recently, deep learning has made significant contributions to challenging tasks, such as machine translation Klimova et al. (2023) and autonomous driving Lee and Liu (2023). In cryptography, machine learning techniques have primarily been applied to practical works in side-channel analysis Masure et al. (2020).

Block ciphers are commonly used in various information systems to ensure confidentiality. One type of cipher that is rapidly gaining popularity is the lightweight block cipher, which is widely employed in Internet of Things (IoT) devices. Lightweight block ciphers are fundamental components of cryptosystems, playing a critical role

*Correspondence:

Nasour Bagheri

Nbagheri@sru.ac.ir

¹ CPS2 Lab, Department of Communication, Faculty of Electrical Engineering, Shahid Rajaee Teacher Training University, Lavizan, Tehran 16788-15811, Iran

² Department of Mathematics, Institute for Advanced Studies in Basic Sciences (IASBS), Zanjan 45137-66731, Iran

³ Electronics Research Institute, Sharif University of Technology, Tehran, Iran

in ensuring data confidentiality in resource-constrained devices. In this context, the primary methods used to attack block ciphers are differential cryptanalysis Biham and Shamir (1991) and linear cryptanalysis Matsui (1993).

Recent advancements in machine learning, especially deep learning, have introduced new paradigms in cryptanalysis. Using neural networks to develop distinguishers has shown promising results in identifying weaknesses in block ciphers. For the first time, at CRYPTO'19, Gohr proposed better results by using a neural network in differential cryptanalysis Gohr (2019). That work focuses on applying deep learning techniques, such as ResNet He et al. (2016), to extend the widely used differential distinguisher model.

In this paper, we extend Gohr's work by introducing a new method for differential neural cryptanalysis based on multiple input polytope differences. Our approach involves using two polytope differences as inputs instead of one. This method allows for distinguishing between the ciphertexts produced by different polytope differences, eliminating the need for random inputs. Furthermore, we utilize a ResNet architecture with dilated convolutional layers and a training method that loads the previous round's model weights at each new round, improving the accuracy of the baseline neural distinguishers for SPECK32, SIMON32 Beaulieu et al. (2015), and SIMECK32 Yang et al. (2015) block ciphers.

Related work

The main results of Gohr's work Gohr (2019) include training neural networks to use the differential features of the SPECK cipher with reduced rounds. For this purpose, neural networks were trained to distinguish SPECK ciphertexts with given input differences from random data. Neural networks have been developed to attack SPECK reduced to eight rounds, and it has been shown that key recovery attacks using neural distinguishers are more optimal than classical methods for nine and eleven rounds.

It is demonstrated that neural distinguishers exploit features of the distribution of ciphertext pairs that are hidden from all purely differential distinguishers, even with unlimited data. Different directions have been explored following the introduction of differential neural cryptanalysis.

Su et al. (2021) proposed a novel approach combining polytope differences with neural networks to improve differential cryptanalysis on reduced-round SIMON32 block cipher. They introduced the Polytope Differential Neural Network Distinguisher, which enhanced the success rate of neural distinguishers from 0.76 to 0.92 on eight rounds of SIMON32. Furthermore, they

developed a key recovery attack leveraging the probability of differential paths alongside their polytope distinguisher, achieving a computational complexity of $2^{33.4}$ for 11-round SIMON32. The Bayesian Key Research with Error was another contribution, reducing the complexity to $2^{30.9}$.

Benamira et al. (2021) investigated the superior performance of neural distinguishers in cryptanalysis, presenting their findings at Eurocrypt 2021. They analyzed Gohr's neural distinguishers and discovered their effectiveness stems from leveraging the differential distribution of ciphertext pairs and distributions in the penultimate and antepenultimate rounds. This dual reliance allows neural distinguishers to surpass traditional differential distinguishers in accuracy. Their study also showed that simplifying Gohr's neural network to basic machine learning tools can still achieve similar accuracy. This suggests that these neural distinguishers effectively approximate the Differential Distribution Table (DDT) during training to classify ciphertext pairs. This insight enhances the interpretability of deep neural networks in cryptanalysis.

Wang et al. (2021) proposed an extended machine learning framework to construct related-key differential distinguishers for lightweight ciphers, such as SPECK32 and PRESENT64. They introduced exhaustive and greedy algorithms for optimizing input and key differences, demonstrating that low Hamming weight differences yield higher accuracy. Using Bayesian optimization, they identified suitable machine learning models, such as CNNs, to improve distinguisher performance. Their methods reduced the data complexity significantly, achieving, for instance, a 7-round related-key distinguisher for SPECK32 with a complexity of 2^8 , compared to $2^{13.2}$ in Gohr (2019).

Hou et al. (2021) proposed improvements to neural distinguishers for SIMON and SPECK by utilizing SAT-based algorithms to optimize input differences and introducing a novel data format leveraging multiple output differences. Their approach significantly enhanced the accuracy and round coverage of neural distinguishers, achieving, for instance, effective 11-round distinguishers for SIMON64 and 8-round distinguishers for SPECK32. The study demonstrated that these advancements enable more efficient distinguishing attacks and reduce complexity in key recovery scenarios.

In Wang et al. (2022), Wang et al. introduce novel multiple differential distinguishers for the round-reduced SIMECK32 cipher based on deep learning. They design two types of distinguishers using neural networks to simulate the multiple input and output differences of differential cryptanalysis. The study presents the general models of these distinguishers and their neural network

structures, utilizing random and associated multiple ciphertext pairs as inputs. Experimental results show that these new distinguishers achieve higher accuracy and can handle more rounds than traditional single-difference distinguishers. The study highlights the efficiency and reduced complexity of deep learning-based differential cryptanalysis.

Lyu et al. (2022a) improved deep-learning-based differential distinguishers by introducing repeated training and testing techniques, significantly enhancing the accuracy and round coverage of neural distinguishers applied to SIMECK32. They extended the NN-based distinguisher from 9 to 10 rounds and the differential-NN distinguisher to 15 rounds. Building on this work, Lyu et al. (2022b) further advanced deep-learning-assisted key recovery attacks for SIMECK32 by focusing on selecting neural distinguishers optimized for Bayesian key search rather than solely prioritizing accuracy. Their approach incorporated precomputed wrong-key response profiles and classical differential transitions with high probabilities and neutral bits, enabling successful key recovery for 13, 14, and 15 rounds of SIMECK32 with reduced data and time complexity compared to prior methods, achieving a success rate of up to 88%.

Bao et al. (2022) enhanced differential-neural cryptanalysis by introducing advanced techniques to extend attacks on round-reduced ciphers like SPECK32 and SIMON32. Their study devised the first practical 13-round key-recovery attack on SPECK32 and improved 12-round attacks by leveraging generalized neutral bits and Bayesian optimization. Additionally, they introduced innovative approaches, including paired and switching differentials, which enabled efficient data reuse and reduced complexity. Experimental results demonstrated significant improvements in both time and data efficiency.

Hambitzer et al. (2023) introduced NNBits, a deep learning ensemble designed for bit-level profiling in cryptographic randomness tests. This tool generalizes neural distinguishers, such as Gohr's, and extends their application to datasets, including avalanche effects from ciphers like SPECK and AES. Their results demonstrate that NNBits match or surpass the performance of traditional statistical tools, such as NIST STS, while providing deeper cryptanalytic insights, including the identification of specific weak bits and an understanding of bias propagation.

Chen et al. (2023) proposed a new neural distinguisher model that leverages derived features from multiple ciphertext pairs for enhanced differential cryptanalysis. Unlike traditional models that rely on single ciphertext pairs, their approach processes groups of $k > 2$ ciphertext pairs, capturing additional features that emerge

from the non-uniform distribution of multiple samples. Experimental results demonstrated improved distinguishing accuracy across five reduced symmetric ciphers, including SPECK32, with significant performance gains over baseline models. Additionally, the proposed model reduced the time complexity of key recovery attacks.

Gohr et al. (2022) assessed various differential-neural distinguishers for multiple ciphers, including SIMON and SPECK, to evaluate their generality and optimization potential. They demonstrated that neural distinguishers can be automatically optimized for improved performance across six ciphers, highlighting a correlation between distinguisher accuracy and distribution skewness. Furthermore, the study rectified misconceptions regarding multi-pair ciphertext distinguishers, showing that claimed improvements were often marginal. Their findings reaffirm the utility of neural distinguishers in cryptanalysis, particularly for scenarios where classical methods are computationally infeasible. However, they emphasize their limitations in leveraging non-differential features.

In Bellini et al. (2023), Bellini et al. develop a general framework for neural cryptanalysis capable of automatically finding suitable input differences and providing general neural distinguishers without human adjustment. Their research comprises two main components: an evolutionary algorithm for searching suitable input differences and the DBitNet architecture, agnostic to the cipher structure. This fully automated pipeline has demonstrated competitive performance with specialized approaches, especially for SPECK32 and SIMON32, and introduced new neural distinguishers for various ciphers (e.g., XTEA, LEA, HIGHT, SIMON128, SPECK128). It also improves the state-of-the-art for PRESENT, KATAN, TEA, and GIMLI. This work lays the groundwork for future research, making it easier to apply neural cryptanalysis to a wider range of ciphers without manual tuning.

In Baksi et al. (2023), Baksi et al. present new advancements in machine learning-based distinguishers, focusing on round-reduced versions of ciphers such as SPECK32, SPECK128, ASCON, SIMECK32, SIMECK64, and SKINNY128. They explore the use of neural networks and support vector machines in various configurations, including different activation functions and multiple input difference tuples. Their results demonstrate the effectiveness of machine learning in cryptanalysis, achieving notable distinguishers for up to 8-round SPECK32 and improved performance for other ciphers. This research highlights the potential of machine learning to enhance classical differential methods, providing a foundation for future studies to further optimize and generalize neural cryptanalysis techniques.

Seok et al. (2024) proposed an innovative method for constructing high-quality differential datasets to enhance the performance of differential neural cryptanalysis. Utilizing Principal Component Analysis (PCA) and K-means clustering, the authors identified input differences that significantly improve the training accuracy of neural distinguishers. Their experiments on lightweight block ciphers SPECK and SIMON demonstrated that datasets optimized using their methodology achieved superior performance compared to traditional approaches, such as Gohr's neural difference search algorithm. The proposed algorithm reduced execution time by 30% while identifying more effective input differences. This study highlights the importance of dataset quality in cryptanalysis and proposes a scalable approach for generating optimized datasets within the context of machine learning-based cryptanalytic techniques.

In Wu et al. (2024), Wu et al. explore the properties of mixture differentials and propose a mixture differential neural network distinguisher using ResNet to improve accuracy in cryptanalysis. Their research focuses on the SIMON32 cipher, demonstrating that the 8-round mixture differential neural network distinguisher achieves an accuracy improvement of 74.7% to 92.3% compared to previous methods. The mixture differential distinguisher is shown to be less sensitive to input differences and offers greater robustness. Additionally, by combining probabilistic expansion and neutral bit techniques, they extend the distinguisher to 11 rounds and perform a successful 12-round key recovery attack on SIMON32 with a key recovery accuracy of 55%, outperforming traditional differential neural network distinguishers.

Contributions

We aim to improve upon the results of Wu et al. (2024), Su et al. (2021), Wang et al. (2022) as well as the baseline differential neural distinguishers. Results are described in Sect. 5, which can be summarized as follows:

- We introduce a data generation method based on multiple input polytope differences and polytopical differential analysis. This method reduces the sensitivity of distinguishers to input differences and eliminates the need for random differences for one of the labels.
- We enhance the accuracy of polytopical, mixture, and multiple differential neural distinguishers for the SIMON32 and introduce the first polytopical differential neural distinguishers for the 32-bit versions of the SIMECK and SPECK ciphers, both in the single-key and related-key scenarios. For SPECK32, our approach achieves 99.39% accuracy for 5 rounds, 91.22% for 6 rounds, 62.01% for 7 rounds, and 50.80% for 8 rounds. For SIMON32, our

method improves the accuracy to 100% for 7 rounds, 98.47% for 8 rounds, and remains effective up to 12 rounds with an accuracy of 50.50%. For SIMECK32, we obtain 100% accuracy for 7 rounds, 98.97% for 8 rounds, and maintain effectiveness up to 12 rounds with an accuracy of 50.55%. These results are further improved in the related-key scenario for SIMON32 and SIMECK32, achieving up to 13 rounds with 50.59% accuracy and 15 rounds with 54.44% accuracy, respectively.

- We conduct a series of experiments to understand the intrinsic performance of neural distinguishers based on polytopical differential analysis. The results of these experiments indicate that the polytopical differential neural distinguisher learns and bases its distinctions on information beyond just the polytopical differential distribution and simple XOR operations. Additionally, ciphertexts provide more details to the distinguisher model than their differences. Furthermore, our distinguishers effectively differentiate between real polytopical differences and randomized differences.

We propose a framework for performing multiple-input polytopical differential neural cryptanalysis of several block ciphers. Our code and trained models are available on GitHub at <https://github.com/NeuralDistinguisher/Multiple-PDND.git>.

Organization

In Sect. 2, we discuss the necessary preliminary concepts, including the structure of the block ciphers under consideration, differential-neural distinguishers, and the idea of polytopical differential analysis. In Sect. 3, we describe our data generation method, which is based on the differences between multiple input polytopes. In Sect. 4, we describe the model architecture used for constructing the distinguishers, along with the training methodology and parameters. In Sect. 5, we present and compare the obtained results with previous works. Additionally, we describe the method for selecting input polytope differences. We also demonstrate the effectiveness of our method through a key recovery test. In Sect. 6, we present the experiments and additional findings. Finally, in Sect. 7, we summarize the paper.

Preliminaries

This section provides an overview of the fundamental terms used in this paper. It includes an introduction to the block ciphers SIMON, SPECK, and SIMECK examined in this work, an explanation of the use of neural networks in differential cryptanalysis, and an overview of polytopical differential analysis.

Short description of SIMON, SPECK, and SIMECK

Lightweight block ciphers SIMON and SPECK, proposed by the U.S. National Security Agency in 2013 Beaulieu et al. (2015), support block sizes ranging from 32 to 128 bits and key sizes from 64 to 256 bits. SIMON employs a Feistel structure with the bitwise AND function as the non-linear operation, while SPECK uses a traditional ARX structure (Addition, Rotation, XOR). SIMON is notable for its simplicity and efficiency in hardware implementations, whereas SPECK, due to its ARX operations, is optimized for software performance on modern processors.

SIMECK, introduced at CHES 2015 Yang et al. (2015), is a family of lightweight block ciphers whose round function and key schedule were inspired by both SIMON and SPECK. Compared to these two ciphers, SIMECK achieves a more compact hardware implementation.

For the 32-bit versions of these ciphers, the round function takes as input two 16-bit words, (x_i, y_i) , and a 16-bit round key, k_i , derived from a 64-bit master key using a key schedule. The next-round state, (x_{i+1}, y_{i+1}) , for SPECK, SIMON, and SIMECK, is computed using the following equations (1), (2), and (3), respectively. Here, $\oplus$ represents the bitwise XOR operation, $\&$ denotes the bitwise AND operation, $\boxplus$ signifies modular addition, and the symbols $\ggg$ and $\lll$ denote circular right and left shifts, respectively.

$$x_{i+1} = ((x_i \ggg 7) \boxplus y_i) \oplus k_i, \quad y_{i+1} = (y_i \lll 2) \oplus x_{i+1} \quad (1)$$

$$x_{i+1} = (x_i \lll 1) \& (x_i \lll 8) \oplus (x_i \lll 2) \oplus k_i, \quad y_{i+1} = x_i \quad (2)$$

$$x_{i+1} = (x_i \lll 5) \& x_i \oplus (x_i \lll 1) \oplus k_i, \quad y_{i+1} = x_i \quad (3)$$

SIMON uses a linear key schedule, whereas SPECK and SIMECK employ their respective non-linear round functions to derive the round keys.

Differential-neural distinguishers

Gohr introduced a neural differential distinguisher for round-reduced SPECK to distinguish ciphertext pairs with a specific input difference from random data Gohr (2019). The model is trained using the following procedures:

- Collecting training data by generating N uniformly distributed keys and plaintext pairs given a fixed input difference and the binary labels Y_i , where i ranges from 0 to $N - 1$.
- If the binary label $Y_i = 1$, then the plaintext pairs are encrypted by r -round SPECK to produce the ciphertext pairs. Otherwise, random ciphertext pairs are generated.

- Pre-process ciphertext pairs to fit the format required by the neural network and start training.

Gohr's work shows that neural networks can be effectively trained to capture the non-randomness of target ciphers. Given a particular input difference Δ_{in} and a target cipher, the neural network is trained to distinguish ciphertext pairs using Δ_{in} from random differences. These ciphertext pairs using Δ_{in} are labeled by 1, and those using random differences are labeled by 0.

Given enough training samples, a deep residual network (ResNet) is trained to distinguish between ciphertext data and random data. The neural network, acting as a distinguisher, outputs a value v ranging from 0 to 1. If $v > 0.5$, the pair of ciphertexts is considered to use Δ_{in} ; otherwise, it is supposed to use a random difference. This trained neural network is referred to as a neural distinguisher (ND).

The effectiveness of neural distinguishers is further demonstrated by their ability to outperform classical differential cryptanalysis methods. For example, Gohr achieved better results than the best classical cryptanalysis on 11-round SPECK, showing the potential of machine learning in cryptanalysis.

Overview of (related-key) polytopic cryptanalysis

Differential cryptanalysis examines how round function iterations transfer plaintext differences to ciphertext differences. Polytopic cryptanalysis Tiessen (2016) is a generalized framework extending traditional differential cryptanalysis by incorporating the interdependencies among larger sets of texts (polytopes) traversing through a cipher. Introduced formally in block cipher security analysis, this method evaluates the statistical relationships of multiple plaintext and ciphertext differences beyond pairwise dependencies. Polytopic transitions enable cryptanalysts to exploit higher-order differential structures and uncover subtle non-random features in cryptographic algorithms that traditional methods might overlook.

A transition involving a $(d + 1)$ -polytope, denoted as $(m_0, m_1, \dots, m_d)$, is described as an s -tuple of values in F_2^n , where differences between texts in the polytope are analyzed relative to a reference text. These differences form the foundation of d -differences:

$$\Delta = (m_0 \oplus m_1, m_0 \oplus m_2, \dots, m_0 \oplus m_d),$$

where m_0 is the anchor or reference text.

Similar to differential cryptanalysis, polytopic trails trace transitions over multiple rounds:

$$\Delta_0 \xrightarrow{f_1} \Delta_1 \xrightarrow{f_2} \dots \xrightarrow{f_r} \Delta_r,$$

with probabilities calculated over all possible intermediate transitions.

Related-key differential cryptanalysis, introduced by Biham in 1991 Biham (1994), extends classical differential cryptanalysis by incorporating differences in the plaintexts and the master keys. In this approach, a pair of plaintexts P and P' is encrypted using a pair of related keys K and K' , respectively, where $K \oplus \Delta_{key} = K'$. The analysis focuses on exploiting differential propagation while encrypting these plaintexts with distinct keys, even if they are identical.

In polytopic differential analysis, this concept is extended to related-key settings by considering $d + 1$ polytopes in both the plaintext and key domains. A related-key polytopic distinguisher uses a $(d + 1)$ -polytope $(m_0, m_1, \dots, m_d)$ for the plaintexts and a corresponding $(d + 1)$ -polytope for the keys $(k_0, k_1, \dots, k_d)$, where $k_0 \oplus \Delta_{key}^1 = k_1, \dots, k_0 \oplus \Delta_{key}^d = k_d$. The relative differences between these texts and keys form the foundation of d -differences:

$$\Delta_P = (m_0 \oplus m_1, m_0 \oplus m_2, \dots, m_0 \oplus m_d),$$

$$\Delta_K = (k_0 \oplus k_1, k_0 \oplus k_2, \dots, k_0 \oplus k_d).$$

This framework analyzes the propagation of polytopic differences through the cipher, represented as an r -round related-key polytopic differential transition. The formal representation of such a transition can be expressed as:

$$(\Delta_P, \Delta_K) \xrightarrow{f} (\Delta'_P, \Delta'_K),$$

where Δ_P and Δ_K denote the input plaintext and key differences, and Δ'_P and Δ'_K represent their corresponding output differences after r rounds.

Dataset generation

The dataset generation process is crucial in training neural network distinguishers for cryptanalysis. This process ensures the model learns to identify non-random characteristics in ciphertext distributions by leveraging structured input differences.

We adopt the multiple input differences method, first introduced by Baksi and Baksi (2022), to construct training data for single-key and related-key scenarios. This method generates plaintext and ciphertext quadruples with specific polytope differences, creating distinguishable patterns for the neural network.

In the single-key scenario, all plaintexts in a quadruple are encrypted with the same random key. In contrast, in the related-key scenario, the plaintexts are encrypted with related keys derived using predefined key differences. Both scenarios use distinct polytope differences for the positive (label 1) and negative (label 0) classes, ensuring structured output distributions that enhance

the neural network's distinguishing capabilities. The dataset generation process comprises the following steps:

1. Random selection of plaintexts and keys.
2. Application of polytope differences to create plaintext quadruples for both label classes.
3. Encryption of the plaintext quadruples using single or related keys to generate ciphertext quadruples.
4. Assignment of labels and storage of the generated data for training purposes.

The following subsections provide a detailed explanation of the dataset generation process for both single-key and related-key scenarios. Subsequently, a structured algorithm is presented to encapsulate the entire procedure.

Single-key scenario

In the single-key scenario, all plaintexts within a quadruple are encrypted using the same randomly selected key, ensuring that the differences in the ciphertext quadruples are determined solely by the differences in the plaintexts. The generated ciphertext quadruples (C^0, C^1, C^2, C^3) and their respective labels Y serve as the input for training the neural network distinguisher.

To construct the dataset, we generate a random plaintext P^0 and apply predefined polytope differences. For label 0, the polytope differences $\Delta = (\Delta^1, \Delta^2, \Delta^3)$ are applied to P^0 , resulting in the plaintext quadruple:

$$(P^0, P^0 \oplus \Delta^1, P^0 \oplus \Delta^2, P^0 \oplus \Delta^3).$$

Similarly, for label 1, the polytope differences $\delta = (\delta^1, \delta^2, \delta^3)$ are applied to generate:

$$(P^0, P^0 \oplus \delta^1, P^0 \oplus \delta^2, P^0 \oplus \delta^3).$$

Each plaintext quadruple is then encrypted using the same randomly selected key K to produce the corresponding ciphertext quadruple:

$$(C^0, C^1, C^2, C^3),$$

where:

$$C^0 = \text{encrypt}(P^0, K), \quad C^j = \text{encrypt}(P^0 \oplus \Delta^j, K) \\ \text{for } j = 1, 2, 3 \quad (\text{label } 0),$$

and:

$$C^0 = \text{encrypt}(P^0, K), \quad C^j = \text{encrypt}(P^0 \oplus \delta^j, K) \\ \text{for } j = 1, 2, 3 \quad (\text{label } 1).$$

The ciphertext quadruples, alongside their labels, form the training dataset for the neural network. The network learns to distinguish between label 0 and label 1 by identifying the statistical differences in the ciphertext

distributions resulting from the input plaintext differences Δ and δ .

The detailed steps for generating the dataset in the single-key scenario are summarized in Algorithm 1.

Algorithm 1 Dataset Generation for Single-Key Scenario

Require: Number of samples N , polytope differences $\Delta = (\Delta^1, \Delta^2, \Delta^3)$, $\delta = (\delta^1, \delta^2, \delta^3)$

Ensure: Training data (X, Y)

```

1:  $X \leftarrow []$ 
2: for  $i \leftarrow 0$  to  $N - 1$  do
3:    $Y_i \xleftarrow{\$}$  random binary label ▷ Assign label 0 or 1 randomly.
4:    $K_i \xleftarrow{\$}$  random 64-bit key ▷ Generate a random encryption key.
5:    $P_i^0 \xleftarrow{\$}$  random plaintext ▷ Generate a random plaintext.
6:   if  $Y_i == 0$  then
7:      $P_i^j \leftarrow P_i^0 \oplus \Delta^j, \quad j = 1, 2, 3$  ▷ Generate plaintext quadruple for label 0.
8:   else
9:      $P_i^j \leftarrow P_i^0 \oplus \delta^j, \quad j = 1, 2, 3$  ▷ Generate plaintext quadruple for label 1.
10:  end if
11:   $C_i^0 \leftarrow \text{encrypt } P_i^0 \text{ with } K_i$ 
12:   $C_i^j \leftarrow \text{encrypt } P_i^j \text{ with } K_i, \quad j = 1, 2, 3$  ▷ Encrypt plaintexts using the same key.
13:  Append  $(C_i^0, C_i^1, C_i^2, C_i^3)$  to  $X$ 
14: end for
15: return  $X, Y$ 

```

Related-key scenario

In the related-key scenario, the plaintext quadruples, and the corresponding encryption keys are structured using predefined polytope differences. This enables a more comprehensive examination of the cipher's behavior under related-key settings, where the encryption keys are not independent but are derived from a master key through specific differences.

Given the input polytope differences $\Delta = (\Delta^1, \Delta^2, \Delta^3)$ for label 0 and $\delta = (\delta^1, \delta^2, \delta^3)$ for label 1, along with key differences $\Delta^{key} = (\alpha^1, \alpha^2, \alpha^3)$ for label 0 and $\delta^{key} = (\beta^1, \beta^2, \beta^3)$ for label 1, the ciphertext quadruples (C^0, C^1, C^2, C^3) and their associated labels Y are generated for training.

Key and Plaintext Generation A random 32-bit plaintext P^0 and a random 64-bit key K are generated. The key K is then split into four parts: $K = \{l_0, l_1, l_2, l_3\}$. Related keys are derived as follows:

- For label 0:

$$K_0^j = \begin{cases} \{l_0, l_1, l_2, l_3 \oplus \alpha^j\} & (\text{SIMON, SIMECK}) \\ \{l_0 \oplus \alpha^j, l_1, l_2, l_3\} & (\text{SPECK}) \end{cases}, \quad j = 1, 2, 3.$$

- For label 1:

$$K_1^j = \begin{cases} \{l_0, l_1, l_2, l_3 \oplus \beta^j\} & (\text{SIMON, SIMECK}) \\ \{l_0 \oplus \beta^j, l_1, l_2, l_3\} & (\text{SPECK}) \end{cases}, \quad j = 1, 2, 3.$$

A plaintext quadruple is generated for each label:

- For label 0:

$$(P^0, P^0 \oplus \Delta^1, P^0 \oplus \Delta^2, P^0 \oplus \Delta^3).$$

- For label 1:

$$(P^0, P^0 \oplus \delta^1, P^0 \oplus \delta^2, P^0 \oplus \delta^3).$$

Ciphertext Generation Each plaintext in the quadruple is encrypted using the corresponding related keys:

- For label 0:

$$C^0 = \text{encrypt}(P^0, K), \quad C^j = \text{encrypt}(P^j, K_0^j), \quad j = 1, 2, 3.$$

- For label 1:

$$C^0 = \text{encrypt}(P^0, K), \quad C^j = \text{encrypt}(P^j, K_1^j), \quad j = 1, 2, 3.$$

The resulting ciphertext quadruples (C^0, C^1, C^2, C^3) , along with their labels, are used as input for the neural network. This structured approach leverages plaintext and key differences to enhance the neural network's ability to identify distinguishing characteristics in ciphertext distributions.

Network architecture

To effectively capture the complex patterns in polytopic differential cryptanalysis, we employ a deep learning model based on the architecture proposed by Bellini et al. (2023). This section outlines the structure of the neural network, which utilizes dilated convolutional layers to enhance feature extraction and to improve model performance. The overall architecture of this model is illustrated in Fig. 1.

Algorithm 2 Dataset Generation for Related-Key Scenario

Require: Number of samples N , polytope differences $\Delta = (\Delta^1, \Delta^2, \Delta^3)$, $\delta = (\delta^1, \delta^2, \delta^3)$, key differences $\Delta^{key} = (\alpha^1, \alpha^2, \alpha^3)$, $\delta^{key} = (\beta^1, \beta^2, \beta^3)$

Ensure: Training data (X, Y)

```

1:  $X \leftarrow []$ 
2: for  $i \leftarrow 0$  to  $N - 1$  do
3:    $Y_i \xleftarrow{\$}$  random binary label ▷ Assign label 0 or 1 randomly.
4:    $K_i \xleftarrow{\$}$  random 64-bit key ▷ Generate a random master key.
5:   Split  $K_i$  into  $\{l_0, l_1, l_2, l_3\}$ 
6:   if  $Y_i == 0$  then
7:      $K_0^j \leftarrow \begin{cases} \{l_0, l_1, l_2, l_3 \oplus \alpha^j\}, & (\text{SIMON, SIMECK}) \\ \{l_0 \oplus \alpha^j, l_1, l_2, l_3\}, & (\text{SPECK}) \end{cases}, \quad j = 1, 2, 3$ 
8:   else
9:      $K_1^j \leftarrow \begin{cases} \{l_0, l_1, l_2, l_3 \oplus \beta^j\}, & (\text{SIMON, SIMECK}) \\ \{l_0 \oplus \beta^j, l_1, l_2, l_3\}, & (\text{SPECK}) \end{cases}, \quad j = 1, 2, 3$ 
10:  end if
11:   $P_i^0 \xleftarrow{\$}$  random plaintext
12:  if  $Y_i == 0$  then
13:     $P_i^j \leftarrow P_i^0 \oplus \Delta^j, \quad j = 1, 2, 3$ 
14:  else
15:     $P_i^j \leftarrow P_i^0 \oplus \delta^j, \quad j = 1, 2, 3$ 
16:  end if
17:   $C_i^0 \leftarrow \text{encrypt } P_i^0 \text{ with } K_i$ 
18:  if  $Y_i == 0$  then
19:     $C_i^j \leftarrow \text{encrypt } P_i^j \text{ with } K_0^j, \quad j = 1, 2, 3$ 
20:  else
21:     $C_i^j \leftarrow \text{encrypt } P_i^j \text{ with } K_1^j, \quad j = 1, 2, 3$ 
22:  end if
23:  Append  $(C_i^0, C_i^1, C_i^2, C_i^3)$  to  $X$ 
24: end for
25: return  $X, Y$ 

```

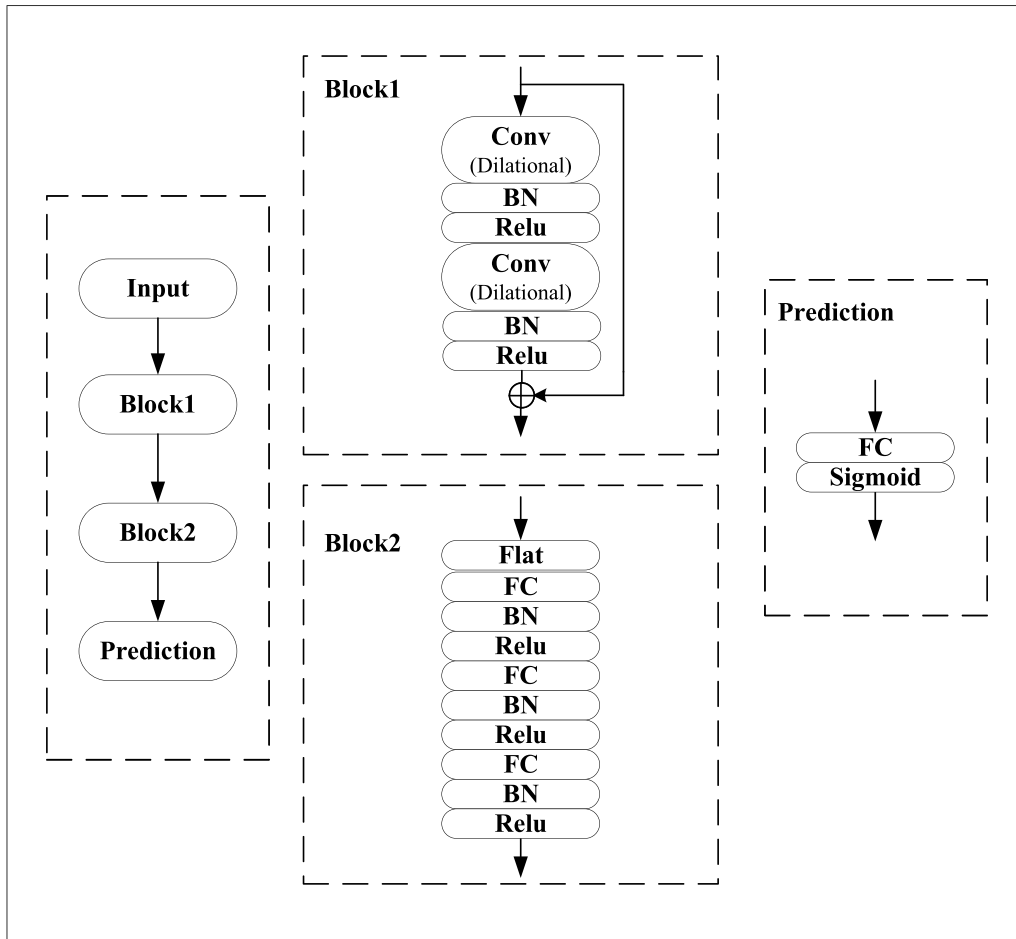


Fig. 1 Architecture of the model

Input and Normalization The input to our model consists of N samples, where the format and structure of each sample is a quadruple of ciphertexts along with their corresponding label, represented as (C^0, C^1, C^2, C^3, Y) . In the main model, each sample from the dataset has four features, each 32 bits in length, and the processing is performed on this input format. The model begins with an input layer where the data is normalized to a range of $[-1, 1]$. This normalization step helps stabilize the training process and enhances the model's convergence. The input size is denoted by $4 \times \text{block size}$, which in this case is 128 bits (since the block size is 32 bits). Using dilated convolutional layers eliminates the need to reshape the input, allowing us to perform bit-oriented analysis directly. This is particularly beneficial for cryptographic analysis, where maintaining the data's bit-level structure is crucial.

Block 1 Block 1 consists of two dilated convolutional layers. The first convolutional layer uses a kernel size of 2, a stride of 1, 'valid' padding, and a dilation rate based on the input size. The second convolutional layer uses a kernel size of 2, a stride of 1, 'causal' padding, and a dilation

rate of 1. Both convolutional layers are followed by batch normalization (BN) and ReLU activation functions. After each pair of convolutional layers, the number of filters is increased by 16. The dilation rates are calculated as follows:

$$\text{dilation_rate}(i) = \left\lfloor \frac{128}{2^i} - 1 \right\rfloor, \quad \text{for } i = 0, 1, 2, \dots,$$

where $\lfloor \cdot \rfloor$ represents the floor function, which returns the greatest integer less than or equal to the input value, $\text{dilation_rate}(i)$ represents the dilation rate for the i -th layer, and 128 is the input size in bits. Dilated convolutions enable an exponential increase in the receptive field without compromising resolution or coverage, making them particularly useful for capturing long-range dependencies in data while maintaining computational efficiency.

Block 2 Block 2 consists of three fully connected (FC) layers, followed by batch normalization (BN) and ReLU activation functions. The first and second dense layers

each have 256 neurons, while the third dense layer has 64 neurons. Additionally, an l_2 regularization with a parameter of 10^{-5} is applied to the dense layers to prevent overfitting.

Prediction Head The prediction head's final layer is a fully connected layer followed by a sigmoid activation function. This layer produces a binary decision, indicating whether the input data conforms to the expected differential pattern.

Despite the inherently random nature of cryptographic algorithms, deep learning models can identify subtle features and patterns that are not directly observable through traditional methods. The model can learn complex representations of the input data by leveraging the power of dilated convolutions and dense layers. These representations capture the underlying relationships between the input differences and the resulting ciphertexts, enabling the model to act effectively. The combination of advanced feature extraction techniques and extensive training on large datasets allows the neural network to detect non-randomness and differentiate between conforming and non-conforming data with high accuracy.

Training distinguishers

The training strategy utilized in this paper adopts a progressive approach. First, a distinguisher for r rounds is trained. The trained weights are then used to initialize the model for the subsequent $(r + 1)$ rounds. This incremental method enhances the model's capability to learn complex patterns effectively across different rounds.

The training continues until the validation accuracy falls below a specified threshold, ten times the standard deviation (10σ) above the random baseline of 50%

accuracy. The standard deviation for the accuracy is determined by the size of the dataset, N , and is given by:

$$\sigma = \frac{1}{2\sqrt{N}}.$$

This criterion ensures that a distinguisher is deemed effective only if its performance exceeds random guessing by a statistically significant margin. During training, the accuracy is carefully monitored for both the training and testing datasets to ensure the reliability and generalization of the distinguisher. The detailed training process is described in Algorithm 3. The same method is used for training and evaluating *RK-PDNDs* in the related-key scenario.

The training process utilizes the Adam optimizer, which features a cyclic learning rate that ranges from 0.001 to 0.0002 over ten epochs. The model is trained for several epochs with early stopping based on validation accuracy. We chose a training set size of 10^7 samples and a validation set size of 10^6 . We ran 120 epochs for each training, with a batch size of 10,000. The loss function used was Mean Squared Error (MSE). Additionally, each model was run 5 times, and the average accuracy of these five runs is reported.

The model training was performed using two Nvidia RTX 3060 Ti GPUs. The programming language used for data generation, model construction, and model training was Python 3.10. The training was executed on a machine with the following specifications: Intel i9-12900K CPU, 16 GB RAM, and Ubuntu LTS 22.04.4 operating system. The machine learning framework utilized was TensorFlow 2, with additional libraries such as NumPy and Pandas.

Algorithm 3 Training *PDNDs*

Require: N (number of training samples), N' (number of validation samples), r_0 (initial round), σ (accuracy standard deviation)

Ensure: Trained *PDND* for the highest non-random round

- 1: **Step 1:** Randomly initialize the neural network *PDND*.
- 2: **Step 2:** Set initial round $r \leftarrow r_0$.
- 3: **Step 3:** Initialize validation accuracy $A_{\text{val}} \leftarrow 1.0$.
- 4: **while** $A_{\text{val}} > 0.50 + 10\sigma$ **do** $\triangleright$ Continue while accuracy is statistically significant
- 5: **Step 4:** Generate the *PDND* training dataset containing N_{train} samples using the appropriate dataset generation algorithm (e.g., Algorithm 1 for single-key or Algorithm 2 for related-key).
- 6: **Step 5:** Generate the *PDND* validation dataset containing N_{val} samples using the same scenario-specific dataset generation algorithm.
- 7: **Step 6:** Train the *PDND* model on the training dataset and its corresponding labels.
- 8: **Step 7:** Measure the validation accuracy A_{val} of the *PDND* model on the validation dataset.
- 9: **Step 8:** Increment the number of rounds $r \leftarrow r + 1$.
- 10: **end while**
- 11: **Step 9:** Return the trained *PDND*.

Testing process

The trained model is evaluated by making predictions on unseen ciphertext quadruples. For each quadruple, the model outputs a prediction score $v \in (0, 1)$, which indicates the likelihood that the ciphertext quadruple is derived from a plaintext quadruple conforming to the input polytopic pattern.

The testing process is conducted on a separate test dataset of 10^6 samples to measure the model's performance. The dataset is composed of ciphertext quadruples generated using two distinct polytopic differences:

- *Polytopic Difference* Δ : Corresponding to label 0.
- *Polytopic Difference* δ : Corresponding to label 1.

For each input quadruple, the model determines whether it was generated using Δ or δ . A higher value of v (closer to 1) indicates the model's confidence in distinguishing between the distributions associated with these two polytopic differences.

Additionally, the model can differentiate between genuine data–ciphertext quadruples generated by encrypting plaintexts using the target block cipher–and random data, which does not conform to any structured polytopic pattern. This demonstrates the model's robustness in identifying structured patterns within ciphertext distributions.

Results

This section presents the results obtained for our distinguishers on the SPECK32, SIMON32, and SIMECK32 ciphers in two scenarios: single-key and related-key. We analyze the performance and accuracy of the neural distinguishers in both settings to evaluate their effectiveness and robustness. In addition, we describe the method for selecting input polytope differences, which significantly improves the accuracy of our neural distinguishers. Furthermore, we demonstrate the applicability of our approach through a key recovery test, where the distinguishers effectively distinguish between correct and incorrect keys.

Single key polytopic differential neural distinguishers

This section compares the neural distinguishers obtained in this work and those from previous research. The results are shown in Table 1.

For our experiments, we used specific polytope differences for each cipher as follows:

For SPECK32, we used the polytope differences

$$\Delta = ((0x40, 0x0), (0x0, 0x8000), (0x60, 0x0))$$

and

$$\delta = ((0x20, 0x0), (0x40, 0x8000), (0x10, 0x2000)).$$

Table 1 Comparison of our single-key polytopic differential neural distinguishers for SPECK32, SIMON32 and SIMECK32 with previous research

Cipher	#R	Acc. (%)	Attack	References
SPECK32	5	99.50	PDND	Sect. 5.1
	6	91.33	PDND	Sect. 5.1
	7	62.01	PDND	Sect. 5.1
	8	51.10	PDND	Sect. 5.1
SIMON32	7	99.60	MDND	Wu et al. (2024)
	7	100	PDND	Sect. 5.1
	8	92.30	MDND	Wu et al. (2024)
	8	98.47	PDND	Sect. 5.1
	9	63.73	PDND	Su et al. (2021)
	9	83.93	PDND	Sect. 5.1
	10	50.17	PDND	Su et al. (2021)
	10	62.78	PDND	Sect. 5.1
	11	53.13	PDND	Sect. 5.1
	12	50.50	PDND	Sect. 5.1
SIMECK32	7	99.98	MDND*	Wang et al. (2022)
	7	100	PDND	Sect. 5.1
	8	96.48	MDND*	Wang et al. (2022)
	8	98.97	PDND	Sect. 5.1
	9	72.48	MDND*	Wang et al. (2022)
	9	86.13	PDND	Sect. 5.1
	10	55.01	MDND*	Wang et al. (2022)
	10	63.18	PDND	Sect. 5.1
	11	53.18	PDND	Sect. 5.1
	12	50.55	PDND	Sect. 5.1
	13	50.11	PDND	Sect. 5.1

PDND, polytopic differential neural distinguisher; MDND, mixture differential neural distinguisher; MDND*, multiple differential neural distinguisher

Table 2 The related-key polytopic differential neural distinguishers for SPECK32, SIMON32 and SIMECK32

Cipher	#R	Acc. (%)	Attack
SPECK32	5	99.50	PDND
	5	99.01	RK-PDND
	6	91.33	PDND
	6	86.40	RK-PDND
	7	62.01	PDND
	7	59.36	RK-PDND
	8	51.10	PDND
	8	50.57	RK-PDND
SIMON32	8	98.47	PDND
	8	100	RK-PDND
	9	83.93	PDND
	9	99.84	RK-PDND
	10	62.78	PDND
	10	96.99	RK-PDND
	11	53.13	PDND
	11	69.32	RK-PDND
	12	50.50	PDND
	12	53.75	RK-PDND
	13	50.59	RK-PDND
SIMECK32	10	63.18	PDND
	10	1	RK-PDND
	11	53.18	PDND
	11	99.94	RK-PDND
	12	50.55	PDND
	12	99.23	RK-PDND
	13	50.11	PDND
	13	84.80	RK-PDND
	14	59.65	RK-PDND
	15	54.44	RK-PDND

PDND, polytopic differential neural distinguisher; RK-PDND, related-key polytopic differential neural distinguisher

For SIMON32 and SIMECK32, we used the polytope differences

$$\Delta = ((0x0, 0x1), (0x0, 0x4), (0x0, 0x8))$$

and

$$\delta = ((0x0, 0x2), (0x0, 0x10), (0x0, 0x40)).$$

The neural distinguishers were trained using the method described in SubSect. 4.1. The Table 1 summarizes the accuracy of the single key distinguishers for different rounds of SPECK32, SIMON32, and SIMECK32. The results show improvements in accuracy compared to previous works. For a fair comparison, we used papers with

similar features, a comparable number of samples, and no staged training or advanced feature engineering.

These results demonstrate that our method, which utilizes two polytope differences and a ResNet-based neural network architecture, significantly improves the accuracy of baseline neural distinguishers for multiple rounds of SPECK32, SIMON32, and SIMECK32 compared to previous methods. Polytope differences reduce the distinguisher model's sensitivity to the specific input difference, making it more robust. Additionally, employing two distinct sets of differences for the positive and negative classes (instead of random differences for the negative class) enhances the separability of the output distributions for both labels, thereby improving the model's ability to distinguish between them.

Related-key polytopic differential neural distinguishers

This subsection presents the results for the related-key polytopic differential neural distinguishers (*RK-PDNDs*) for SPECK32, SIMON32, and SIMECK32. The analysis was conducted similarly to the single-key scenario, using the same parameters but with related keys. The results are summarized in Table 2.

For SPECK32, the input key differences used were

$$\Delta^{key} = (0x0040, 0x8000, 0x0060),$$

$$\delta^{key} = (0x0020, 0x8000, 0x2000).$$

For SIMON32, the input key differences were

$$\Delta^{key} = (0x0, 0x8000, 0x0), \quad \delta^{key} = (0x0, 0x8000, 0x2000).$$

For SIMECK32, the input key differences were

$$\Delta^{key} = (0x1, 0x4, 0x8), \quad \delta^{key} = (0x0020, 0x0010, 0x0040).$$

The input polytope difference values Δ and δ for the plaintexts are the same as the values mentioned in SubSect. 5.1.

The results in Table 2 show that for SPECK32, the number of rounds distinguished by the *RK-PDND* is similar to that of the *PDND*. Still, the accuracy in the related-key scenario is generally lower. This indicates the resistance of SPECK's key schedule algorithm to this analysis, which does not provide additional features or information to the model in the related-key scenario. However, for SIMON32 and SIMECK32, the number of rounds distinguished by the *RK-PDND* is higher than that of the *PDND*, and the accuracy for all rounds in the related-key scenario is significantly better.

When the number of rounds distinguished for SIMON32 and SIMECK32 is compared, we observe that the number of rounds and accuracy in the single key scenario are almost similar. However, in the related-key scenario, SIMECK32 shows more distinguishable rounds and better accuracy than SIMON32. This suggests that SIMON32's linear key schedule provides more resistance against related-key polytope differential neural analysis compared to the non-linear key schedule of SIMECK32, which uses its round function for key distribution.

On the choice of polytope input differences

The selection of polytope input differences plays a crucial role in the effectiveness of our neural distinguishers. A well-chosen polytope difference can significantly enhance the performance of the distinguisher by optimizing its ability to identify distinguishing features across cipher rounds. In this section, we discuss the importance of polytope input differences, the criteria for selecting suitable differences, the methodology used in this study, and the impact of these choices compared to previous work, particularly Baksi and Baksi (2022).

The primary factor influencing the selection of polytope input differences is minimizing the Hamming weight of the differences. Lower Hamming weight differences reduce the dispersion of the difference across the cipher rounds, making it easier for the neural distinguisher to identify and classify the distribution of differences. This is because differences with high Hamming weights tend to spread more widely, making them less predictable and more challenging to track through the rounds of the cipher. By keeping the Hamming weight

Table 3 Comparison of the performance of different polytope input differences for 6-round SPECK32 and 8-round SIMON32

Cipher	#R	Δ	δ	Acc. (%)
SPECK32	6	((0x40, 0x0), (0x0, 0x8000), (0x60, 0x0))	((0x20, 0x0), (0x40, 0x8000), (0x10, 0x2000))	91.33
		((0x40, 0x0), (0x8000, 0x0), (0x20, 0x0))	((0x100, 0x0), (0x40, 0x0), (0x1, 0x0))	91.00
		((0x0, 0x1), (0x0, 0x4), (0x0, 0x8))	((0x0, 0x20), (0x0, 0x10), (0x0, 0x40))	89.40
		((0x0, 0x10), (0x0, 0x8000), (0x0, 0x400))	((0x100, 0x0), (0x40, 0x0), (0x2, 0x0))	88.99
		((0x0, 0x10), (0x0, 0x6000), (0x0, 0x400))	((0x100, 0x8), (0x40, 0x0), (0x2, 0x0))	83.34
		((0x0, 0x10), (0x0, 0x8), (0x10, 0x0))	((0x100, 0x40), (0x40, 0x8), (0x1, 0x2000))	54.30
		((0x0, 0x40), (0x3000, 0x0), (0x0, 0x6))	((0x100, 0x6), (0x40, 0x8), (0x2, 0x3))	50.00
		((0x0, 0x1), (0x0, 0x4), (0x0, 0x8))	((0x0, 0x2), (0x0, 0x10), (0x0, 0x40))	98.47
SIMON32	8	((0x0, 0x1), (0x0, 0x3), (0x0, 0x6))	((0x0, 0x2), (0x8, 0x10), (0x1, 0x40))	94.71
		((0x20, 0x1), (0x0, 0x4), (0x0, 0x100))	((0x2, 0x0), (0x0, 0x80), (0x0, 0x10))	93.80
		((0x0, 0x80), (0x0, 0x100), (0x0, 0x8))	((0x0, 0x20), (0x0, 0x40), (0x0, 0x10))	93.71
		((0x1, 0x0), (0x4, 0x0), (0x8, 0x0))	((0x0, 0x2), (0x0, 0x10), (0x0, 0x40))	86.80
		((0x0, 0x4002), (0x0, 0x4001), (0x0, 0x2001))	((0x3, 0x0), (0x2, 0x0), (0x1, 0x0))	60.99
		((0x1, 0x8000), (0x4000, 0x2), (0x200, 0x8))	((0x40, 0x80), (0x200, 0x4), (0x1000, 0x20))	59.61

low, the propagation of the difference across rounds is more concentrated, which leads to a more effective distinguisher.

Moreover, the placement of the active bit in the input difference is also critical. For SIMON-like ciphers (SIMON and SIMECK), placing the active bit in the right word (i.e., in the form $(0, e_i)$) was found to yield better results. For the SPECK cipher, the placement of the active bit in either of the two words can yield effective results; however, typically placing it in the left word produces better outcomes for the neural distinguisher. This strategic placement of active bits ensures that the differences propagate efficiently, optimizing the distinguisher's ability to detect them.

In this work, the polytope input differences were selected through an extensive experimental evaluation of various potential differences. We tested several polytope differences, adjusting their Hamming weights and the placement of active bits, and selected the differences that provided the best performance for each cipher. This process allowed us to identify the most effective polytope differences for SPECK32, SIMON32, and SIMECK32. It is essential to note that the differences chosen in this study differ from those used in previous work, particularly Baksi and Baksi (2022). While the method presented in Baksi and Baksi (2022) relies on pure differential analysis, our approach employs polytopical differential analysis, which allows for greater flexibility in selecting input differences and thus enables better optimization of the neural distinguisher's performance. For example, the best neural distinguisher reported in Baksi et al. (2023) as an extension of the work in Baksi and Baksi (2022) for the SIMECK32 cipher was achieved for 9 rounds, whereas we extend the analysis to 13 and 15 rounds, demonstrating the robustness of our method in handling more complex cases.

In this study, various polytope input differences were evaluated to determine their impact on the performance of the neural distinguisher. Based on the findings and discussions provided in this section, Table 3 presents a comparison of the performance and effect of several polytope input differences on the neural distinguishers for 6-round SPECK32 and 8-round SIMON32. The table highlights the influence of different input differences on the accuracy of the distinguisher.

In conclusion, the choice of polytope input differences is a critical factor in optimizing the performance of neural distinguishers. By minimizing the Hamming weight and carefully selecting the placement of active bits, we improved the effectiveness of the distinguisher for SIMON-like and SPECK ciphers. The flexibility in choosing polytope differences, combined with polytopical differential analysis, has enabled better performance compared

to previous approaches, as demonstrated by our experiments and the results presented in this paper.

Key recovery test

This section presents a key recovery test to evaluate the ability of our neural distinguishers to distinguish between correct and incorrect subkeys. The primary goal is to assess the potential of our distinguishers in recovering the correct subkey through partial decryption and to demonstrate their applicability in cryptanalysis.

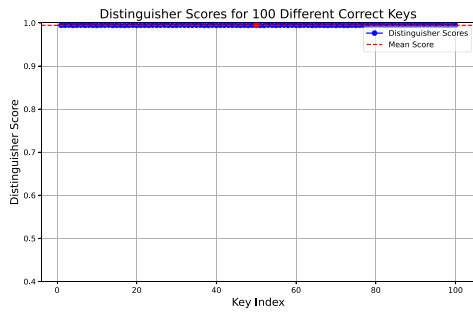
To show the effectiveness of the proposed neural distinguishers as research tools, we developed a partial key recovery attack utilizing the r -round distinguishers for the lightweight block ciphers SIMON32, SIMECK32, and SPECK32. In this approach, we focus on evaluating the performance of the neural network model trained using an r -round distinguisher for each of the target encryption algorithms. The model is tested to see how accurately it can predict the correct subkey as compared to incorrect ones when extended to $r + 1$ rounds.

The key recovery test is performed as follows:

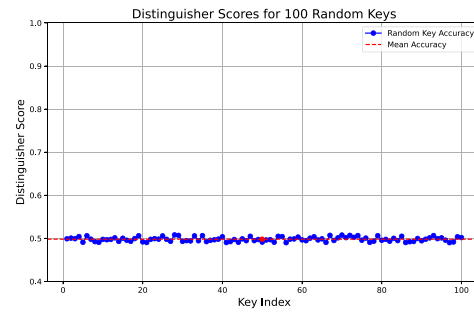
1. After training the neural network model, we select 10^6 plaintext quadruple samples. A fixed key is applied to all these samples.
2. The corresponding ciphertext quadruples are generated using $r + 1$ rounds of the encryption algorithm.
3. These $r + 1$ round ciphertext quadruples are then partially decrypted using both the correct key and a set of random incorrect keys, with one round of decryption applied in each case.
4. For each resulting ciphertext quadruple, input features for the neural network model are generated as described in Sect. 3.
5. The accuracy of the model is then evaluated based on its ability to distinguish the correct key from incorrect ones.

This process is repeated across 100 different random keys, and the average accuracy is calculated. The expected result is that, for the correct key, the model's accuracy should closely match the accuracy achieved with the r -round distinguisher. For incorrect keys, the accuracy should be approximately 0.5, which is close to random guessing.

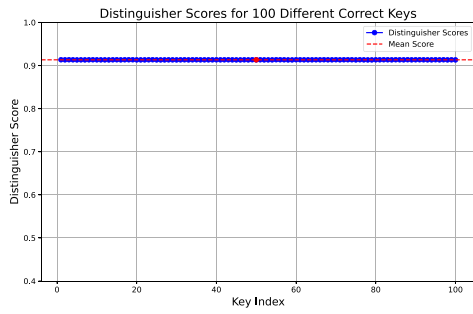
For the single key experiments involving SIMON32, SIMECK32, and SPECK32, we applied the key recovery test for $r + 1$ rounds ranging from $r = 5$ to $r = 8$ for SPECK32 and $r = 8$ to $r = 11$ for SIMECK32 and SIMON32, based on the distinguishers and their accuracies presented in Table 1. Additionally, we extended this key recovery test to related-key scenarios for $r + 1$



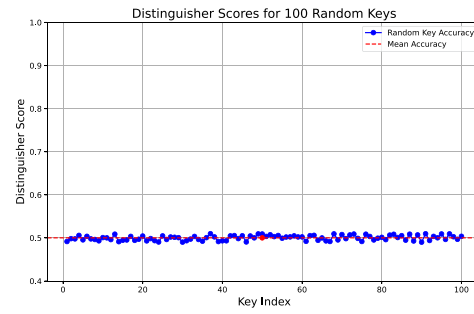
(a) 6-rounds (Correct Keys)



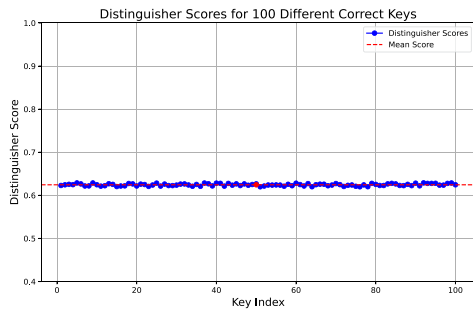
(b) 6-rounds (Random Keys)



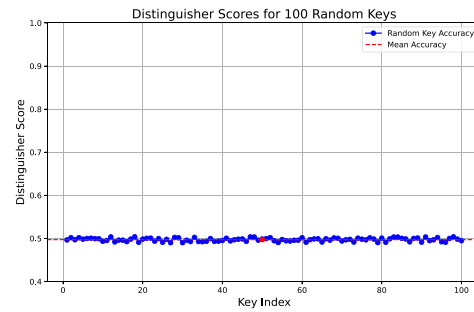
(c) 7-rounds (Correct Keys)



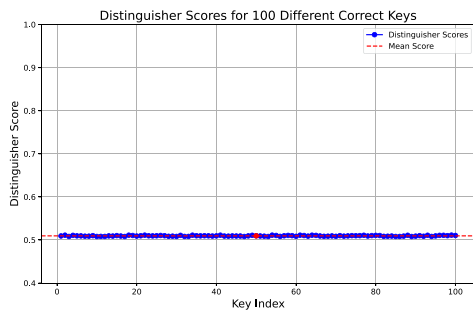
(d) 7-rounds (Random Keys)



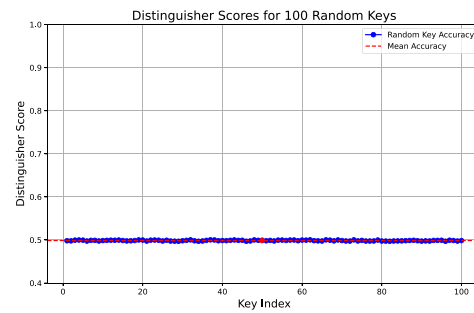
(e) 8-rounds (Correct Keys)



(f) 8-rounds (Random Keys)

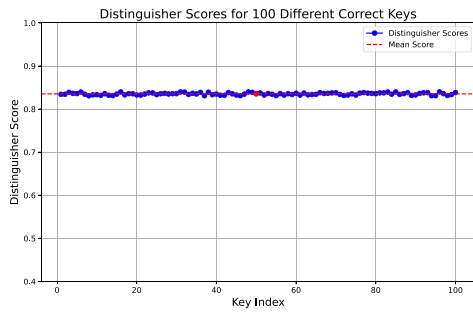


(g) 9-rounds (Correct Keys)

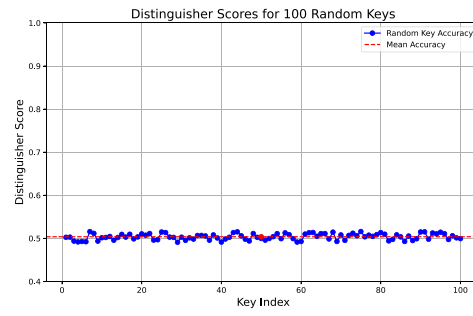


(h) 9-rounds (Random Keys)

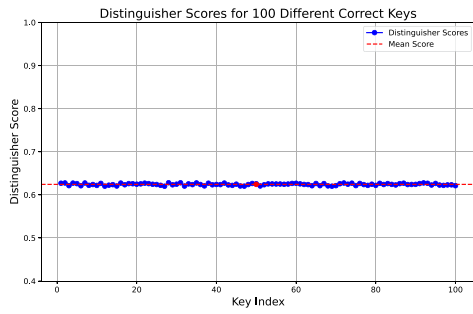
Fig. 2 PDND scores for single-key key recovery tests across 6 to 9 rounds of SPECK32



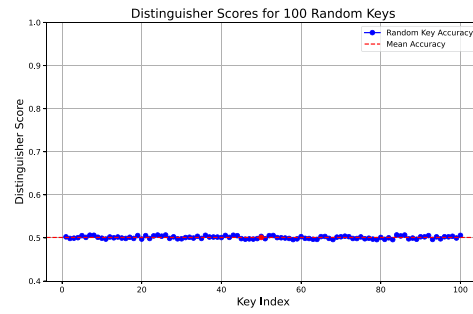
(a) 10-rounds (Correct Keys)



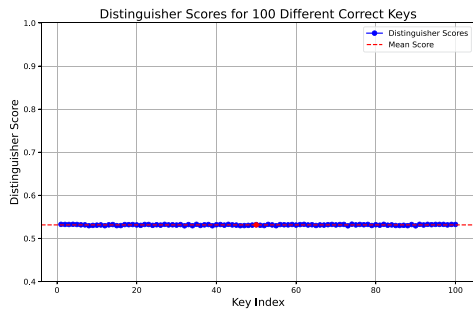
(b) 10-rounds (Random Keys)



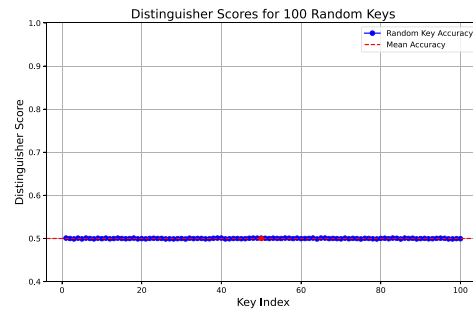
(c) 11-rounds (Correct Keys)



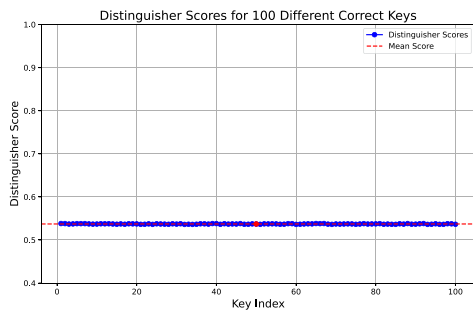
(d) 11-rounds (Random Keys)



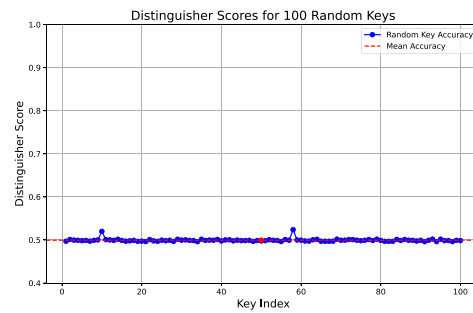
(e) 12-rounds (Correct Keys)



(f) 12-rounds (Random Keys)

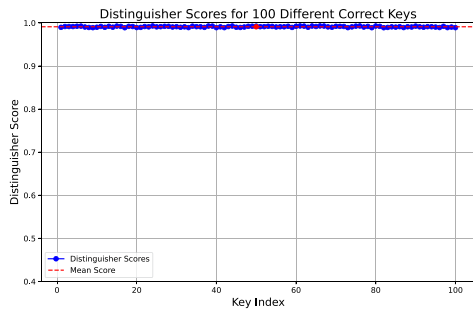


(g) 13-rounds (Correct Keys)

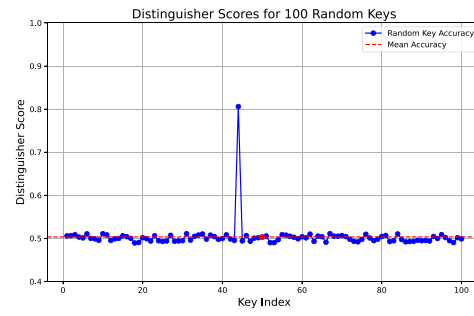


(h) 13-rounds (Random Keys)

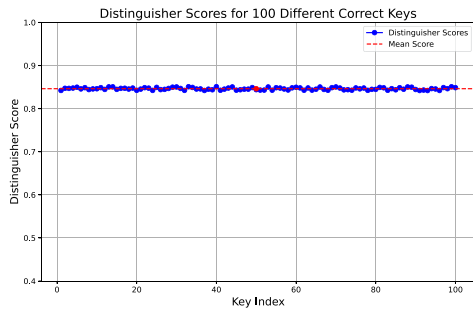
Fig. 3 PDND scores for single-key key recovery tests across 10 to 12 rounds of SIMON32 and 13 rounds for related-key



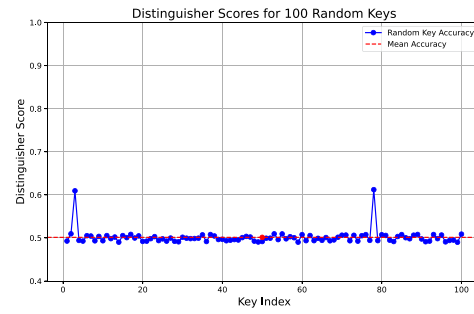
(a) 13-rounds (Correct Keys)



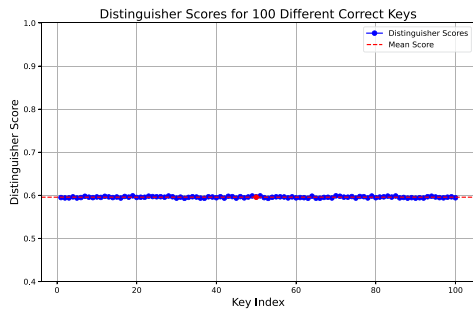
(b) 13-rounds (Random Keys)



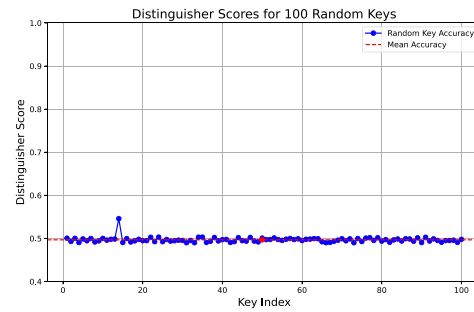
(c) 14-rounds (Correct Keys)



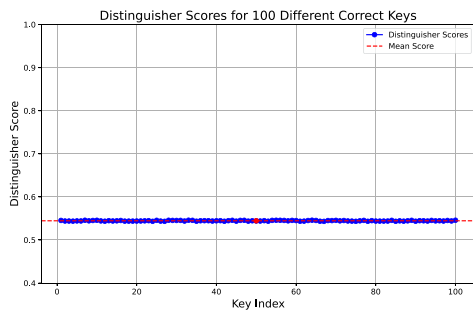
(d) 14-rounds (Random Keys)



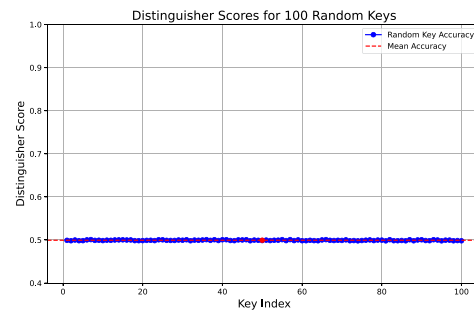
(e) 15-rounds (Correct Keys)



(f) 15-rounds (Random Keys)



(g) 16-rounds (Correct Keys)



(h) 16-rounds (Random Keys)

Fig. 4 PDND scores for related-key key recovery tests across 13 to 16 rounds of SIMECK32

Table 4 Summary of deep learning-based attacks on target algorithms

Cipher	Attack model	#R	Acc. (%)	N	N'	Key recovery	SR (%)	Time	Data	References
SPECK32 –	DND	3	79	2^{25}	2^{21}	6+3+1	–	–	–	Yadav and Kumar (2021)
	DND	6	68.8	10^7	10^6	–	–	–	–	Ebrahimi et al. (2022)
	DND	7	59.10	10^7	10^6	7+1	92	2^{16}	2^8	Yue and Wu (2023)
	DND	7	61.6	10^7	10^6	1+7+1	70	2^{16}	2^8	Gohr (2019)
	DND	7	61.6	10^7	10^6	1+2+7+1	52.10	2^{38}	$2^{14.5}$	Gohr (2019)
	DND	7	66.94	16×10^7	16×10^6	1+2+7+1	59.5	2^{36}	$2^{14.12}$	Chen et al. (2023)
	DND	8	57.74	32×10^7	32×10^6	–	–	–	–	Liu et al. (2023)
	PDND	8	51.10	10^7	10^6	1+8+1	100 [†]	–	–	Sect. 5.4
	NASA	8	51.83	10^7	10^7	2+8	75.51	2^{16}	$2^{14.65}$	Chen et al. (2023)
SIMON32	DND	5	57	2^{25}	2^{22}	7+5+1	–	–	–	Yadav and Kumar (2021)
	MDND	8	92.30	10^7	10^6	1+2+8+1	55	$2^{31.19}$	2^{18}	Wu et al. (2024)
	DND	9	60	10^7	10^6	1+9+1	92	$2^{18.5}$	2^9	Hou et al. (2021)
	PDND	9	63.73	10^7	10^6	1+9+1	100	$2^{30.9}$	2^{14}	Su et al. (2021)
	DND	9	60	10^7	10^6	1+2+9+1	90	$2^{18.5}$	$2^{12.5}$	Hou et al. (2021)
	DND	9	63.20	10^7	10^6	1+4+9+1	23	$2^{20.1}$	2^8	Tian and Hu (2021)
	DND	11	56.16	32×10^7	32×10^6	–	–	–	–	Liu et al. (2023)
	PDND	11	53.13	10^7	10^6	1+11+1	100 [†]	–	–	Sect. 5.4
	DND	11	51.74*	2^{30}	2^{28}	1+3+11+1	49.99	$2^{44.2}$	2^{21}	Bao et al. (2022)
	DND	11	58.71	16×10^7	16×10^6	1+4+11+1	9	$2^{54.01}$	2^{28}	Zhang et al. (2024)
	RK-PDND	12	53.75	10^7	10^6	1+12+1	100 [†]	–	–	Sect. 5.4
SIMECK32	DND	7	28.0	10^4	10^4	–	–	–	–	Jain et al. (2021)
	DND	9	52.60	2^{15}	2^{15}	–	–	–	–	Baksi et al. (2021)
	DND	9	52.60	2^{15}	2^{15}	–	–	–	–	Baksi et al. (2023)
	MDND*	9	72.48	2^{24}	2^{18}	–	–	–	–	Wang et al. (2022)
	DND	10	54.07	10^7	10^6	–	–	–	–	Lyu et al. (2022a)
	DND	10	54.21	10^7	10^6	1+3+10+1	88	$2^{33.90}$	2^{24}	Lyu et al. (2022b)
	PDND	11	53.18	10^7	10^6	1+11+1	100 [†]	–	–	Sect. 5.4
	DND	12	51.61*	16×10^7	16×10^6	1+3+12+1	30	$2^{45.04}$	2^{26}	Zhang et al. (2023)
	RK-PDND	15	54.44	10^7	10^6	1+15+1	100 [†]	–	–	Sect. 5.4

[†]Reported values are obtained from the execution of the key recovery test presented in SubSect. 5.4. * The staged training method is used to train the ND model (higher data complexity).

N , training data complexity; N' , testing data complexity; SR, success rate for key recovery; Time, time (computational) complexity; Data, data complexity; DND, differential neural distinguisher; NASA, neural-aided statistical attack; PDND, polytopic differential neural distinguisher; RK-PDND, related-key polytopic differential neural distinguisher; MDND*, mixture differential neural distinguisher; MDND*, multiple differential neural distinguisher; –, key recovery not attempted or not reported

rounds, using related-key distinguishers for SIMON32, SIMECK32, and SPECK32. Notably, these included distinguishers for up to 12 rounds of SIMON32, and up to 12 to 15 rounds of SIMECK32, as shown in Table 2. The results for distinguishers with accuracy below 51% were excluded due to their proximity to random guessing.

Figures 2, 3, and 4 present the model's accuracy in distinguishing between correct and incorrect subkeys for both single-key and related-key scenarios. The left columns of the figures show the accuracy when the correct keys are used, while the right columns illustrate

the performance with random keys. Specifically, Fig. 2 focuses on the 6 to 9 rounds of SPECK32 in the single-key scenario, Fig. 3 covers 10 to 13 rounds of SIMON32 for both single-key and related-key and Fig. 4 demonstrates the results for 13 to 16 rounds of SIMECK32 in the related-key scenario. As demonstrated in these figures, our proposed neural network model effectively distinguishes between correct and incorrect keys for $r + 1$ rounds, showcasing its capability in key recovery tasks.

As illustrated in the corresponding figures, we observe that as the number of rounds increases, the accuracy of the distinguisher in differentiating between the correct key and incorrect keys tends to decrease. This trend is expected, since the cryptographic structure becomes more complex and the statistical differences between correct and incorrect keys become less pronounced in higher rounds. However, this reduction in accuracy can be partially compensated by increasing the number of samples and utilizing larger, more diverse datasets for both training and evaluation. Additionally, as the number of samples increases, the average score assigned by the distinguisher to the correct key becomes closer to the neural distinguisher's accuracy. In practical key recovery attacks as well, the lower accuracy of the distinguisher can generally be compensated by increasing the data complexity Chen et al. (2023). Conversely, reducing the number of dataset samples leads to a decrease in the distinguisher's accuracy, and consequently, a lower neural distinguisher score for the correct key.

In comparison to the key recovery results for 11-round and 12-round SIMON32 presented in Su et al. (2021) and Wu et al. (2024), our distinguishers, without the complexity of adding additional rounds, have been extended to 12 rounds in the single-key scenario and 13 rounds in the related-key scenario, as explained above. Furthermore, the key recovery presented in Wang et al. (2022) for SIMECK32 was for 11 rounds, but in this section, using our *PDND* distinguishers, it has been extended to 12 rounds in the single-key scenario and up to 16 rounds in the related-key scenario.

In addition, leveraging the first subkey addition after the initial nonlinearity application, all distinguishers discussed in this paper can be extended by one additional round at no extra cost. In a chosen-plaintext scenario, an adversary can easily introduce plaintext polytope differences into the output of the first round of SIMON, SIMECK, or SPECK, making it possible to apply the key recovery attack for $1 + r + 1$ rounds with the *PDNDs*.

Based on this, Table 4 provides a comparative analysis of deep learning-based attacks on SPECK32, SIMON32, and SIMECK32. Rows highlighted in Bold indicate distinguishers with higher data complexity, which are not directly comparable. Works that did not perform key recovery are marked with “-”, and certain studies (e.g., Lu et al. 2024; Ebrahimi et al. 2023; Wang et al. 2024) are excluded due to their incomparable complexity and lack of key recovery evaluation.

Table 4 demonstrates that our proposed neural distinguishers not only achieve high accuracy in distinguishing correct and incorrect keys but also enable competitive or improved attacks compared to prior works. Additionally, by identifying a suitable s -round polytopic differential

characteristic with a high probability (where its output difference matches the input difference of the r -round neural distinguisher) and leveraging neutral bits Bao et al. (2022) to generate ciphertext structures, this characteristic can be prepended to the neural distinguisher. This extends the total attack rounds to $1 + s + r + 1$, further enhancing the cryptanalytic reach.

Computational complexity in key recovery attacks is typically measured by the number of required encryption (and decryption) operations. In contrast, data complexity refers to the number of plaintext samples utilized during the attack. In practical attacks, when the neural distinguisher achieves high accuracy, the correct key is often recovered rapidly, and not all theoretically required data is necessarily used Zhang et al. (2024). The success rate (SR) in key recovery experiments can be defined based on different criteria. For example, in Zhang et al. (2023), the success rate is defined as the ratio of successfully recovered subkeys to the total number of experiments. An experiment is considered successful when the overall SR reaches 99%, i.e., for an experiment count n_e that satisfies $1 - (1 - \text{SR})^{n_e} = 0.99$.

In our key recovery tests, the SR can be defined as the fraction of times the neural distinguisher distinguishes the correct key from incorrect keys, or as the fraction of times the mean distinguisher score for the correct key matches the accuracy of the neural distinguisher. Using the first metric, we observed an SR of 100%; using the second metric, and with a standard deviation of 0.03, the SR remains at 100%. Moreover, as the difference between

Table 5 Accuracy of neural distinguishers trained with output polytope differences ($C^0 \oplus C^1, C^0 \oplus C^2, C^0 \oplus C^3$)

Cipher	#R	Acc. (%)
SPECK32	5	99.46
	6	89.96
	7	61.02
	8	51.02
SIMON32	7	99.66
	8	86.04
	9	65.15
	10	54.52
	11	50.73
SIMECK32	12	50.24
	7	99.91
	8	92.27
	9	67.51
	10	54.83
	11	51.04
	12	50.33
	13	50

the mean output scores for the correct and incorrect keys increases, the SR also improves. In general, increasing computational complexity will further enhance the SR in practical attacks Chen et al. (2023).

In contrast to traditional differential cryptanalysis, neural network-based methods require only the input difference, allowing them to capture a wider spectrum of output differences and thereby enabling more effective key recovery attacks Hou et al. (2021).

On another note, based on the findings in Baksi et al. (2021), it may also be feasible to recover (part of) the round key using the linear inequality generated by the Support Vector Machine (SVM), instead of the neural network. In summary, this section highlights that our *PDNDs* are valuable tools for cipher designers to evaluate security margins and empower attackers to mount key recovery attacks. The results underscore the dual utility of neural distinguishers in both defensive and offensive cryptographic research.

Experiments

In this section, we conduct experiments to evaluate the intrinsic performance and learning mechanisms of neural distinguishers based on polytopic differential analysis. Additionally, we explore methods to improve their accuracy. The experiments are conducted in a single-key scenario.

Training with output polytope differences

In the first set of experiments, we modify the training data to use output polytope differences instead of ciphertexts. Specifically, we calculate the polytope differences of the ciphertexts and use these differences as input to the neural distinguishers. The input data format for the model becomes $(C^0 \oplus C^1, C^0 \oplus C^2, C^0 \oplus C^3)$ along with their corresponding labels, where $\oplus$ denotes the XOR operation. This approach allows the model to focus on the polytopic differential properties of the ciphertexts.

For each sample, we generate four plaintexts (P^0, P^1, P^2, P^3) and encrypt them to obtain the ciphertexts (C^0, C^1, C^2, C^3) based on Algorithm 2. The input polytope differences are similar to those in SubSect. 5.1. The output polytope differences are then calculated as $(C^0 \oplus C^1, C^0 \oplus C^2, C^0 \oplus C^3)$.

The results of this experiment are summarized in Table 5. Based on the results in Table 1 and the comparison with using ciphertext inputs, it is evident that the accuracy of neural distinguishers decreases when the input is limited to the output polytope differences rather than the full ciphertexts.

For the SPECK32 cipher, the accuracy of distinguishers trained on output polytope differences shows a significant decrease compared to those trained on ciphertexts. For instance, the 7-round accuracy drops from 62.01% to 61.02%. Similarly, for the SIMON32 cipher, the accuracy for nine rounds drops from 83.93% to 65.15%. For the SIMECK32 cipher, the 9-round accuracy drops from 86.13% to 67.51%.

This reduction in accuracy indicates that when the model is restricted to the polytope differences of the four ciphertexts and does not have access to the full ciphertexts, its ability to distinguish between samples decreases. The number of rounds that can be accurately distinguished also decreases by at least one round for SIMON32 and SIMECK32 ciphers.

These findings suggest that the ciphertext quadruples give the model more features and information than the output polytope differences. Similar to differential neural distinguishers, the *PDND* can extract more valuable features from the ciphertexts, which are not limited to the differential properties but also include relationships between the bits of the ciphertexts and linear approximations of them, as well as information from the penultimate and antepenultimate rounds of the encryption algorithm Benamira et al. (2021). This additional information allows the model to achieve higher accuracy and distinguish more rounds than using only the polytope differences.

Applying input differences from classical mixture differential characteristics

To investigate whether the neural distinguisher can achieve better accuracy using the input difference from the r -round mixture differential path, we compare its performance with classical mixture input differences used in previous research. Specifically, we utilize the characteristic from the paper Qiao et al. (2024) for the SIMON32 cipher.

According to Table 10 in Qiao et al. (2024), the input mixture difference determined for the 17-round distinguisher of SIMON32 is:

Table 6 Comparison of neural distinguishers' performance with different input differences

Cipher	#R	Acc. (%)	Input difference source
SIMON32	7	66.89	Classical MD Qiao et al. (2024)
	7	99.6	MDND Wu et al. (2024)
	7	100	PDND (Sect. 5)
	8	54.81	Classical MD Qiao et al. (2024)
	8	92.3	MDND Wu et al. (2024)
	8	98.47	PDND (Sect. 5)

$$\Delta = ((0x0001, 0x1150), (0x0100, 0x0444), (0x0101, 0x4104)).$$

In this experiment, we use this value as the input polytope difference for our neural distinguisher and generate the corresponding ciphertexts for label 1 using this input difference. The ciphertexts corresponding to label 0 will have random differences.

Table 6 compares the results obtained from this experiment with those from Wu et al. (2024) and the single-key distinguishers obtained in our study. As shown in Table 6, the results for the neural distinguishers trained with the input difference from Qiao et al. (2024) appear to be less satisfactory, with significantly lower accuracy and performance compared to the differential neural distinguishers trained with the input differences from Wu et al. (2024) and our study.

Based on the test results, it seems that the polytope differential neural distinguisher does not perform as well using the input difference from the r -round mixture differential path. However, it is important to note that the classical characteristic may not be the most suitable for this task, and other factors might contribute to the observed performance.

Additionally, while one of the advantages of *PDNDs* is their lower sensitivity to input differences compared to differential neural distinguishers, it remains essential to choose appropriate differences. It appears that the selected differences, such as those in differential neural distinguishers, should have a Hamming weight of 1 to ensure effective results. Despite the inherent robustness of *PDNDs*, arbitrary differences may not yield satisfactory performance.

Real polytope differences experiment

Inspired by Aaron Gohr's experiments on neural differential distinguishers, we will perform a similar experiment to investigate how well our neural distinguishers can differentiate between real and randomized polytope differences.

Previous work has shown that neural distinguishers trained on differential properties can effectively recognize reduced-round versions of block ciphers. These experiments generated a set of ciphertext pairs, and half of them were randomized by applying a blinding value.

The neural distinguishers were then tasked with distinguishing between the real and randomized pairs. The results demonstrated that the neural distinguishers could identify real differences without explicit training for this task, leveraging inherent features within the data Gohr (2019).

We aim to replicate and extend this experiment using our polytopic differential analysis approach. We will generate a set of ciphertext quadruples (C^0, C^1, C^2, C^3) and then randomize half using a random blinding value R . The neural distinguishers will be tasked with classifying these quadruples as either real or random. This experiment will help us determine whether our polytopic differential neural distinguishers can extract additional information from the ciphertexts, achieving similar or better distinguishing accuracy compared to the baseline differential neural distinguishers.

Experimental steps

- **Data Generation:** Generate 10^6 samples of ciphertext quadruples (C^0, C^1, C^2, C^3) using the polytopic differential method described earlier (Algorithm 1). The input polytope differences are aligned with those presented in SubSect. 5.1.
- **Randomization:** Apply a random blinding value R to half of the samples to produce randomized ciphertext quadruples $(C^0 \oplus R, C^1 \oplus R, C^2 \oplus R, C^3 \oplus R)$.
- **Classification:** Use pre-trained neural distinguishers to classify the ciphertext quadruples as real or random based on the polytope differences.
- **Evaluation:** Measure the accuracy of the neural distinguishers in distinguishing between real and randomized samples.

By conducting this experiment, we aim to validate whether the neural distinguishers based on polytopic differential analysis can effectively identify real differences and leverage additional features from the ciphertexts, similar to the findings in Gohr (2019). Given the similar behavior of distinguishers for the SIMON and SIMECK ciphers, we will conduct this experiment on the 5-round SPECK and 7-round SIMON ciphers.

The results of this experiment indicate that, despite not being specifically trained for this task, the polytopic differential neural distinguishers can effectively distinguish between the generated samples. This demonstrates that the distinguishers can learn information beyond a simple XOR and identify more complex patterns. The results of this experiment are also shown in Table 7.

Table 7 Results of neural distinguishers on real polytope differences for 5-round SPECK and 7-round SIMON

Cipher	#R	Accuracy (%)
SPECK32	5	96.38
SIMON32	7	98.45

Discussion on utilizing advanced methods for further enhancing PDNDs

This subsection discusses various methods for transforming our basic neural distinguishers into advanced models with improved performance and accuracy. These methods are primarily based on selecting an optimal neural network, optimizing its parameters, and feature engineering.

The first method for enhancing the performance of distinguishers involves selecting and employing a better, often more complex, neural network. The most crucial aspect of this task is optimizing the hyperparameters of the network according to the specific task assigned to the model. In addition, choosing a more sophisticated training method, such as staged training Gohr (2019), can also lead to higher accuracy for the neural distinguishers.

The second category of methods for improving the performance of neural distinguishers is related to data generation and feature engineering techniques. Among these, the most critical aspect is the input data format to the model and its features. The more these features include information about the structure of the target cipher algorithm, the better the performance of the neural distinguisher. These features can include, for instance, information from the penultimate rounds of the cipher algorithm, which can be added to the input samples. For example, one can compute the right-branch input differences of the last round of the SPECK cipher without involving the subkey and provide this to the model.

Multiple ciphertexts per sample can further enrich the input data and improve model performance. In Chen et al. (2023), Chen and Yu developed neural distinguisher models for cryptanalysis that utilize features derived from multiple pairs of ciphertexts. This new model utilizes non-uniform distributions to generate derived features that are absent in single ciphertext pairs. The neural distinguishers achieve higher distinguishing accuracy by incorporating these derived features than baseline models. Experiments demonstrate that this approach significantly enhances distinguishing capability and improves key recovery attacks.

Therefore, the performance of basic neural distinguishers can generally be significantly improved by employing different models and optimizing their parameters following cryptanalysis and feature engineering, considering the structure of the cipher algorithm and the analysis method.

Conclusion

In this paper, we introduced Multiple Input Polytopic Differential Neural Distinguishers (PDNDs) for the SIMON, SIMECK, and SPECK block ciphers, leveraging

deep learning techniques and the ResNet architecture to enhance the effectiveness of neural distinguishers. By utilizing multiple polytope differences for data generation, our approach significantly improved the ability of the distinguishers to detect specific differential patterns in ciphertexts. Experimental results in both single-key and related-key scenarios demonstrated that PDNDs consistently outperformed baseline differential neural distinguishers in terms of accuracy.

Furthermore, through extensive experimentation, we examined the intrinsic behaviors and performance characteristics of our neural distinguishers. These investigations allowed us to address various aspects, including enhancing previous results, extending the attack to previously untested ciphers, and analyzing the underlying mechanisms of the distinguishers.

Our findings underscore the potential of advanced neural network architectures combined with multiple input polytope differences as powerful tools in cryptanalysis. Future research could explore the application of these techniques to other cryptographic primitives, develop more sophisticated models, automate the search for optimal polytope differences, and extend the use of deep learning to classical cryptanalytic attacks, thereby unlocking further opportunities in this domain.

Acknowledgements

We would like to thank the anonymous reviewers for their valuable comments and suggestions, which have greatly improved the content and presentation of this paper.

Author contributions

The paper was the result of collaborative efforts between Iman Mirzaali, Sadegh Sadeghi, and Nasour Bagheri. I.M., as the primary student contributor, handled most of the simulations and data analysis. S.S. and N.B. provided essential guidance on the theoretical aspects of the project and validated the results. All authors contributed to the initial manuscript draft and proofread the final version before submission.

Funding

This work is based upon research funded by Iran National Science Foundation (INSF) under project No. 4026806. Nasour Bagheri acknowledges the financial support received from Shahid Rajaee Teacher Training University under grant number 1404/385248.

Data availability

The developed codes and trained models are available on GitHub: <https://github.com/NeuralDistinguisher/Multiple-PDND.git>.

Declarations

Competing interests

The authors declare no conflict of interest. The founding sponsors had no role in the design of the study; in the collection, analysis, or interpretation of data; in the writing of the manuscript, and in the decision to publish the results.

Received: 19 December 2024 Accepted: 1 September 2025

Published online: 19 May 2026

References

- Klimova B, Pikhart M, Benites AD, Lehr C, Sanchez-Stockhammer C (2023) Neural machine translation in foreign language teaching and learning: a systematic review. *Educ Inf Technol* 28(1):663–682
- Lee D-H, Liu J-L (2023) End-to-end deep learning of lane detection and path prediction for real-time autonomous driving. *SIIP* 17(1):199–205
- Masure L, Dumas C, Prouff E (2020) A comprehensive study of deep learning for side-channel analysis. *IACR Trans Cryptograph Hardw Embedded Systd* 2020:348–375
- Biham E, Shamir A (1991) Differential cryptanalysis of des-like cryptosystems. *J Cryptol* 4:3–72
- Matsui M (1993) Linear cryptanalysis method for des cipher. In: *Workshop on the Theory and Application of Cryptographic Techniques*, Springer, pp. 386–397
- Gohr A (2019) Improving attacks on round-reduced speck32/64 using deep learning. Springer International Publishing-In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part II, vol. 39, pp. 150–179
- He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778
- Beaulieu R, Shors D, Smith J, Treatman-Clark S, Weeks B, Wingers L (2015) The simon and speck lightweight block ciphers. In: *Proceedings of the 52nd Annual Design Automation Conference*, pp. 1–6
- Yang G, Zhu B, Suder V, Aagaard MD, Gong G (2015) The simeck family of lightweight block ciphers. In: *International Workshop on Cryptographic Hardware and Embedded Systems*, Springer, pp. 307–329
- Su H-C, Zhu X-Y, Ming D (2021) Polytopic attack on round-reduced simon32/64 using deep learning. In: *Information Security and Cryptology: 16th International Conference, Inscrypt 2020, Guangzhou, China, December 11–14, 2020, Revised Selected Papers*, Springer, pp. 3–20
- Benamira A, Gerault D, Peyrin T, Tan QQ (2021) A deeper look at machine learning-based cryptanalysis. In: *Advances in Cryptology—EUROCRYPT 2021: 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17–21, 2021, Proceedings, Part I 40*, Springer, pp. 805–835
- Wang G, Wang G, He Y (2021) Improved machine learning assisted (related-key) differential distinguishers for lightweight ciphers. In: *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, IEEE, pp. 164–171
- Hou Z, Ren J, Chen S (2021) Improve neural distinguishers of Simon and speck. *Secur Commun Netw* 2021(1):9288229
- Wang H, Tian J, Zhang X, Wei Y, Jiang H (2022) Multiple differential distinguisher of simeck32/64 based on deep learning. *Secur Commun Netw* 2022(1):7564678
- Lyu L, Tu Y, Zhang Y (2022) Improving the deep-learning-based differential distinguisher and applications to simeck. In: *2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, IEEE, pp. 465–470
- Lyu L, Tu Y, Zhang Y (2022) Deep learning assisted key recovery attack for round-reduced simeck32/64. In: *International Conference on Information Security*, Springer, pp. 443–463
- Bao Z, Guo J, Liu M, Ma L, Tu Y (2022) Enhancing differential-neural cryptanalysis. In: *International Conference on the Theory and Application of Cryptology and Information Security*, Springer, pp. 318–347
- Hambitzer A, Gerault D, Huang YJ, Aaraj N, Bellini E (2023) Nnbits: Bit profiling with a deep learning ensemble based distinguisher. In: *Cryptographers' Track at the RSA Conference*, Springer, pp. 493–523
- Chen Y, Shen Y, Yu H, Yuan S (2023) A new neural distinguisher considering features derived from multiple ciphertext pairs. *Comput J* 66(6):1419–1433
- Gohr A, Leander G, Neumann P (2022) An assessment of differential-neural distinguishers. *Cryptology ePrint Archive*
- Bellini E, Gerault D, Hambitzer A, Rossi M (2023) A cipher-agnostic neural training pipeline with automated finding of good input differences. *IACR Trans Symmetric Cryptol* 2023(3):184–212
- Baksi A, Breier J, Dasu VA, Hou X, Kim H, Seo H (2023) New results on machine learning-based distinguishers. *IEEE Access* 11:54175–54187
- Seok B, Chang D, Lee C (2024) A novel approach to construct a good dataset for differential-neural cryptanalysis. *IEEE Trans Depend Secur Comput* 22(1):246–262
- Wu Z, Qiao K, Wang Z, Cheng J, Zhu L (2024) Mixture differential cryptanalysis on round-reduced simon32/64 using machine learning. *Mathematics* 12(9):1401
- Tiessen T (2016) Polytopic cryptanalysis. In: *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8–12, 2016, Proceedings, Part I 35, Springer, pp. 214–239
- Biham E (1994) New types of cryptanalytic attacks using related keys. *J Cryptol* 7:229–246
- Baksi A, Baksi A (2022) Machine learning-assisted differential distinguishers for lightweight ciphers. *Classical and Physical Security of Symmetric Key Cryptographic Algorithms*, 141–162
- Chen Y, Shen Y, Yu H (2023) Neural-aided statistical attack for cryptanalysis. *Comput J* 66(10):2480–2498
- Lu J, Liu G, Sun B, Li C, Liu L (2024) Improved (related-key) differential-based neural distinguishers for simon and simeck block ciphers. *Comput J* 67(2):537–547
- Ebrahimi A, Gerault D, Palmieri P (2023) Deep learning-based rotational-xor distinguishers for and-rx block ciphers: Evaluations on simeck and simon. In: *International Conference on Selected Areas in Cryptography*, Springer, pp. 429–450
- Wang G, Wang G, Sun S (2024) Investigating and enhancing the neural distinguisher for differential cryptanalysis. *IEICE Trans Inf Syst* 107(8):1016–1028
- Zhang L, Wang Z, Wang B (2024) Improving differential-neural cryptanalysis. *IACR Commun Cryptol* 1(3)
- Zhang L, Lu J, Wang Z, Li C (2023) Improved differential-neural cryptanalysis for round-reduced simeck32/64. *Front Comp Sci* 17(6):176817
- Hou Z, Ren J, Chen S (2021) Cryptanalysis of round-reduced simon32 based on deep learning. *Cryptology ePrint Archive*
- Baksi A, Breier J, Dasu VA, Dong X, Yi C (2021) Following-up on machine learning assisted differential distinguishers. In: *SILC Workshop—Security and Implementation of Lightweight Cryptography*, vol. 3, p. 12
- Yadav T, Kumar M (2021) Differential-ml distinguisher: Machine learning based generic extension for differential cryptanalysis. In: *International Conference on Cryptology and Information Security in Latin America*, Springer, pp. 191–212
- Ebrahimi A, Regazzoni F, Palmieri P (2022) Reducing the cost of machine learning differential attacks using bit selection and a partial ml-distinguisher. In: *International Symposium on Foundations and Practice of Security*, Springer, pp. 123–141
- Yue X, Wu W (2023) Improved neural differential distinguisher model for lightweight cipher speck. *Appl Sci* 13(12):6994
- Liu J, Ren J, Chen S, Li M (2023) Improved neural distinguishers with multi-round and multi-splicing construction. *J Inf Secur Appl* 74:103461
- Tian W, Hu B (2021) Deep learning assisted differential cryptanalysis for the lightweight cipher simon. *KSII Trans Internet Inf Syst (TIIS)* 15(2):600–616
- Jain A, Kohli V, Mishra G (2021) Deep learning based differential distinguisher for lightweight block ciphers. *arXiv preprint arXiv:2112.05061*
- Qiao K, Wu Z, Cheng J, Ou C, Wang A, Zhu L (2024) Bit-wise mixture differential cryptanalysis and its application to Simon. *IEEE Internet Things J* 11(13):23398–23409

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.